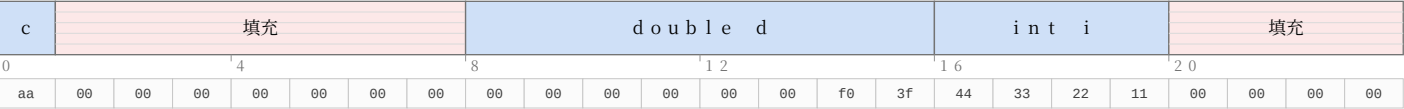


p1_02_layout.cpp · 内存结构图

按本机实测绘制（Apple clang 21 / arm64 / macOS）：1 格 = 1 字节。上排是字段归属，中间是偏移标尺，下排是实跑 dump 出的原始字节。

数据成员 填充（内容未定义） 库类型内部 标志位 / 指针字段

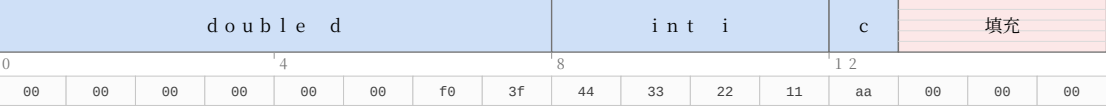
LooseLayout char → double → int（顺序不好） sizeof = 24 字节



下排 = 本例 dump 出的原始字节（小端；?? = 填充内容未定义，实测每次运行都不同）

- double 要求 8 对齐：char 占偏移 0，double 只能从偏移 8 开始，中间 7 字节变成填充；int 落在偏移 16，尾部再补 4 → 24。
- 字节全对得上：aa 是 char；f0 3f 是 double 1.0 的高位（小端）；44 33 22 11 是 int 0x11223344。
- 这里的填充显示为 00，是因为 LooseLayout 是平凡类型，`l{}` 值初始化会连填充位一起清零；换成含非平凡成员的 With String，{} 就不再清零填充位 —— 下一页就是反例。

TightLayout double → int → char（调整后） sizeof = 16 字节



下排 = 本例 dump 出的原始字节（小端；?? = 填充内容未定义，实测每次运行都不同）

- 同样三个成员：8 + 4 + 1 + 3 = 16；填充被挤到尾部，只剩 3 字节。
- 规则：成员按自身对齐落位，结构体整体按最大成员对齐 —— 把大的放前面、小的挤到最后，就能把填充集中到尾部一块。

同样的三个成员，只改声明顺序

LooseLayout（char 打头） 24 字节

TightLayout（double 打头） 16 字节

- 同样三个成员、同样的值，只改声明顺序：24 → 16，省 8 字节（约 33%）。
- Rust 默认会重排字段来省内存，C++ 不会：手动排序是 C++ 程序员的基本功。

三条规则

1 . 成员按自身对齐要求落位：char = 1、int = 4、double = 8、指针 = 8；放不下就往后挪，空洞变成填充。

2 . 结构体整体按最大成员对齐，尾部常要补一段，保证数组里每个元素都对齐（LooseLayout 是 8 对齐、TightLayout 也是 8 对齐）。

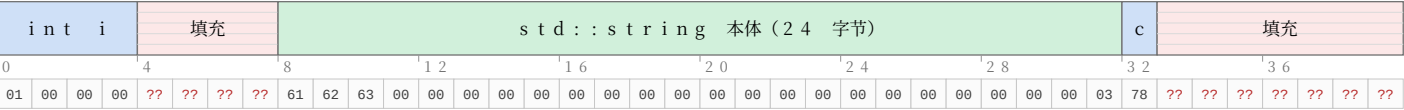
3 . 填充属于对象的一部分（sizeof 算进去），但它的内容未定义：不要 memcmp 比较含填充的结构体，也不要按字节序列化后跨平台传 —— 这正是宿主层与前端交换数据时要先定协议的原因。

p1_02_layout.cpp · 内存结构图（续）

WithString 把「库类型」也放进来：std::string 的本体大小是固定的，但字符串数据在哪，取决于长度。

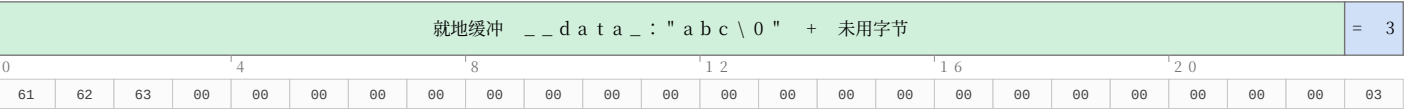
数据成员 填充（内容未定义） std::string 本体 指针 / 标志字段

WithString int + std::string + char sizeof = 40 字节



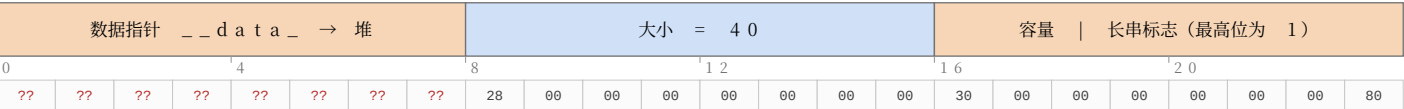
下排 = 本例 dump 出的原始字节（小端；?? = 填充内容未定义，实测每次运行都不同）
· 4 + 4 + 24 + 1 + 7 = 40。std::string 要 8 对齐，所以 i 之后空出 4 字节；c 之后为对齐到 8 的倍数再补 7 字节。
· 算出来的是「本体 24 字节」，不是「字符串 3 字节」—— sizeof 是类型合同，数组步长、跨语言传参的 ABI 都按它走。
· 偏移 4...7 和 33...39 是本例里唯一没被写过的位置：多次运行值都不一样，所以图上记 ??

std::string 本体 · 短串（SSO） len ≤ 22：数据就放在对象里 sizeof = 24 字节



下排 = 本例 dump 出的原始字节（小端；?? = 填充内容未定义，实测每次运行都不同）
· 实测：字符串本体 24 字节 = 23 字节缓冲 + 1 字节长度；缓冲起始地址就等于对象自身地址，没有任何堆分配。
· libc++ 采用「数据在前」布局：最后 1 字节是长度，len=22 时它等于 0x16；超过 22 字符（len=23）就整体切换成长串模式。

std::string 本体 · 长串（堆） len ≥ 23：对象里只留指针 sizeof = 24 字节



下排 = 本例 dump 出的原始字节（小端；?? = 填充内容未定义，实测每次运行都不同）
· 实测 len=40：size=40（0x28）、capacity=47、data 在堆上（指针值每次运行都不同，图上记 ??）；容量字段去掉最高位是 0x30 = 48 = 容量 47 + 结尾 '\0'。
· 所以「std::string 占 24 字节」这句话在两种模式下都成立，但「字符串数据在哪」完全不同 —— 这就是例题 2-1 的伏笔。

读这张图要记住的三条

1. 填充字节的内容是未定义的。本机实测：WithString 的填充每次运行都不一样；把缓冲区先填成 0xAB、再 placement new 构造对象，构造后填充位置仍是 ab —— 证明编译器根本不会去写它。

2. std::string 本体固定 24 字节，数据的位置随长度变：≤ 22 字符就地存放（SSO，零堆分配），≥ 23 字符变成「指针 + 大小 + 容量」三个 8 字节字段。

3. WithString 的 40 = 4(int) + 4(填充) + 24(string 本体) + 1(char) + 7(尾部填充)，与例题的 dump 逐字节对得上：字段值能认出来，填充位认不出来。